



TicketStadio

Sistema di vendita biglietti calcio

Relazione di progetto — Ingegneria del Software

Giacomo Panareo · Università degli Studi di Ferrara

A.A. 2025/2026

Oggetto del progetto

Cosa fa TicketStadio

TicketStadio è una web application per la vendita online di biglietti per partite di calcio di uno specifico stadio. Gli utenti registrati possono consultare il calendario delle partite, scegliere un posto numerato sulla mappa del settore, applicare codici promozionali e finalizzare l'acquisto. È previsto un pannello amministrativo per la gestione di partite, promozioni e statistiche di vendita.

Stack tecnologico

Backend: Java 17, Spring Boot 3.2, JPA, JWT

Database: MySQL 8.0 (InnoDB, utf8mb4)

Frontend: HTML5, CSS3, JavaScript vanilla

Test: JUnit 5, Mockito, H2 in-memory



Utenti target

TIFOSO

Acquista biglietti singoli, sottoscrive abbonamenti, applica promozioni.

ADMIN

Gestisce calendario partite, crea codici sconto, monitora le vendite.

Distribuzione del team

Progetto individuale — tutti i ruoli ricoperti dallo studente



Product Owner

Analisi requisiti, scrittura user stories, priorità del backlog.



Sviluppatore

Implementazione backend Spring Boot, frontend vanilla JS, schema DB.



QA Tester

Scrittura test JUnit, definizione casi limite, verifica copertura.



DevOps

Setup ambiente, Dockerfile, gestione repository Git, dump del DB.

Nota: il lavoro è stato organizzato in giornate dedicate (analisi/PO, implementazione, testing) per simulare i diversi ruoli.

Evidenze delle cerimonie

Processo Scrum adattato a progetto individuale



Sprint Planning

Inizio settimana

Definizione obiettivi della settimana e selezione card da "Backlog" a "In progress".



Daily (self)

Ogni giorno

Auto-review di 5 minuti: cosa ho fatto ieri, cosa farò oggi, blocchi attivi.



Sprint Review

Fine settimana

Verifica funzionalità completate, demo personale, validazione contro user stories.



Retrospective

Fine sprint

Note in file SPRINT_LOG.md: cosa ha funzionato, cosa migliorare, lezioni apprese.

Evidenza pratica: il file **SPRINT_LOG.md** nel repository registra le retrospettive di ogni sprint, con tag #fatto, #problema, #lezione.

Disegno funzionale

Flusso utente: dall'arrivo all'acquisto biglietto (estratto da Miro)

Eccezioni gestite:

posto occupato → 409 Conflict ·

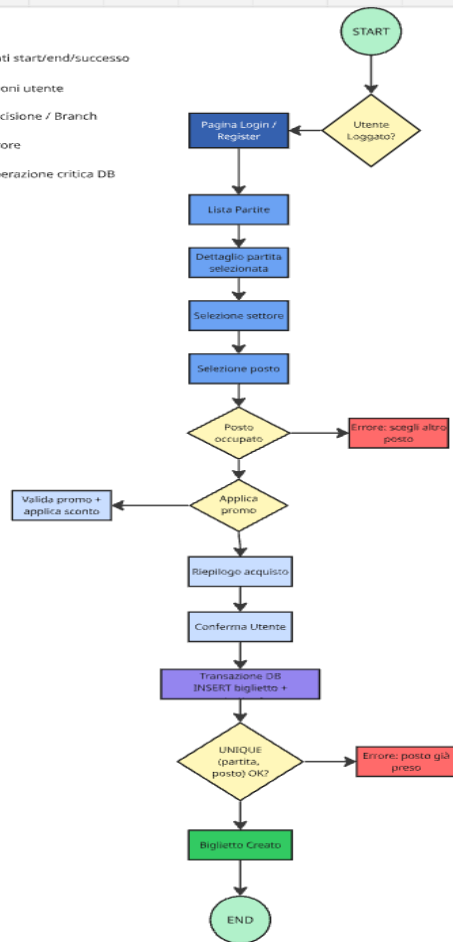
partita annullata → blocco ·

promo scaduta → errore.

FLUSSO ACQUISTO BIGLIETTO

LEGENDA:

- Stati start/end/successo
- Azioni utente
- Decisione / Branch
- Errore
- Operazione critica DB



Board Trello

TicketStadio — Sprint Backlog

Backlog 8 →

- Mappa interattiva 3D dello stadio
- Notifiche push partite preferite
- Supporto multistadio
- Sistema rating partite
- Wallet utente per cashback
- Integrazione pagamento reale
-

Da fare 3 →

- Dashboard admin con grafici
- Refactor CSS in moduli
- Documentazione API in italiano

In esecuzione 0 →

Fatto 25 →

- Setup progetto Spring boot + Maven
- Schema database MySQL(10 tabelle)
- Entità JPA e repository
- Login utente con JWT
- Registrazione nuovo utente
- API CRUD partite (admin)

Test cases

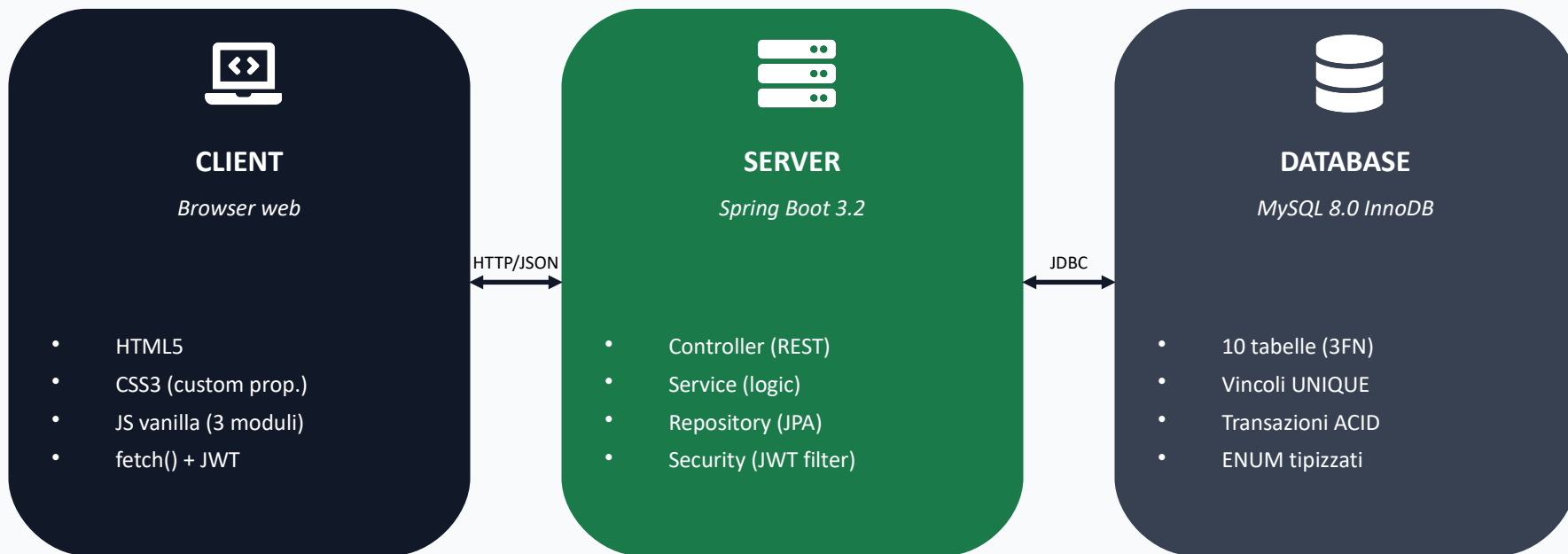
23 test su 3 livelli — eseguiti con mvn test

Unit Test11 test	Slice Test7 test	Integration Test5 test
<i>JUnit 5 + Mockito</i> BigliettoServiceTest, PromozionelsValidaTest. Isolano il service mockando i repository. • Acquisto senza promo • Acquisto con sconto 10% • Posto occupato → eccezione • Partita annullata → blocco • Promo scaduta → rifiuto	<i>@DataJpaTest + H2</i> RepositoryTest. Avvia H2 in-memory, verifica le query JPA contro schema reale. • findByEmail trovato • existsByEmail • Posto libero/occupato • findPostiOccupatiByPartita • findByCodice promo	<i>@SpringBootTest + MockMvc</i> AuthControllerTest. Avvia il contesto Spring completo, simula richieste HTTP reali. • POST /register → 201 • POST /register email dup → 400 • POST /login valido → JWT • POST /login pwd errata → 401 • Validazione password corta

```
$ cd backend && mvn test → Tests run: 23, Failures: 0, Errors: 0, Skipped: 0
```

Disegno architetturale

Architettura a 3 livelli con pattern MVC e Repository



Pattern adottati: MVC server-side · Service Layer · Repository · DTO · Dependency Injection · REST resource-oriented · Stateless authentication.

Funzionalità implementate

Elenco completo delle feature consegnate

TIFOSO (7 funzionalità)

-  Registrazione + Login JWT
-  Visualizzazione partite (filtri)
-  Acquisto biglietto con posto numerato
-  Applicazione codici promozionali
-  Sottoscrizione abbonamenti stagionali
-  Rinnovo abbonamento stagione successiva
-  Storico personale dei biglietti

ADMIN (7 funzionalità)

-  Dashboard statistiche vendite
-  CRUD completo delle partite
-  Annullamento partita + invalidazione biglietti
-  Creazione e gestione promozioni
-  Generatore codici promo random
-  Gestione settori dello stadio
-  Autorizzazione role-based (@PreAuthorize)

Dettaglio: acquisto biglietto + feedback

Funzionalità rappresentativa e considerazioni finali



Acquisto biglietto

Flow tecnico (BigliettoService.acquista):

1. JwtFilter valida il token e popola SecurityContext
2. @PreAuthorize verifica autenticazione
3. @Valid valida il body della request
4. Service apre transazione (@Transactional)
5. Controllo disponibilità posto (existsByPartitaAndPosto)
6. Applicazione promo (se presente e valida)
7. INSERT biglietto + INSERT pagamento atomici
8. **Vincolo UNIQUE DB blocca race condition → 409**



Feedback esperienza

Cosa ha funzionato

Iniziare con lo schema DB ha guidato l'intera architettura.
Trello ha evitato lo scope creep.

Difficoltà incontrate

Bilanciare REST e UX: alcune azioni naturali per l'utente non mappavano subito su una risorsa.

Cosa farei diversamente

Iniziare prima i test (TDD). Aggiungere Liquibase per le migrazioni DB versionate.

Grazie per l'attenzione · github.com/giacomopanareo/ticketstadio · Aperto a domande